

Machine Learning — Complete End-to-End Cheat Sheet

50% Theory · 50% Code | Python · scikit-learn · PyTorch · NumPy · Pandas

Table of Contents

1. ML Fundamentals
 2. Data Preprocessing
 3. Supervised Learning
 4. Unsupervised Learning
 5. Model Evaluation & Metrics
 6. Feature Engineering
 7. Regularization
 8. Ensemble Methods
 9. Neural Networks & Deep Learning
 10. Natural Language Processing (NLP)
 11. Hyperparameter Tuning
 12. End-to-End ML Pipeline
-

1. ML Fundamentals

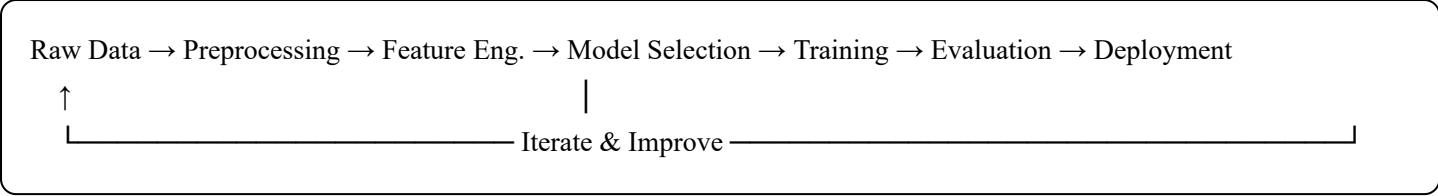
Theory

Machine Learning is the science of getting computers to learn from data without being explicitly programmed.

Three Main Types:

MACHINE LEARNING			
SUPERVISED	UNSUPERVISED	REINFORCEMENT	
(Labeled Data)	(No Labels)	(Reward Signal)	
• Classification	• Clustering	• Q-Learning	
• Regression	• Dimensionality Reduction	• Policy Gradient	
	• Actor-Critic		
	• Anomaly Det.		

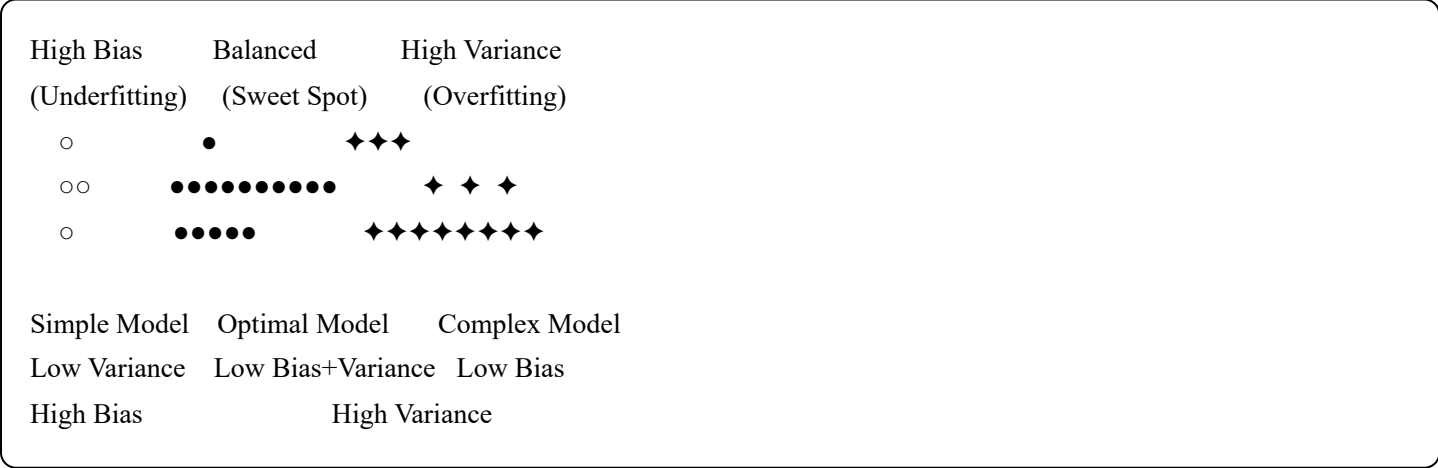
The ML Workflow:



Key Terminology:

Term	Definition
Feature (X)	Input variable used to make predictions
Label/Target (y)	Output we want to predict
Training Set	Data used to train the model (~70-80%)
Validation Set	Data used to tune hyperparameters (~10-15%)
Test Set	Final evaluation data, never touched until end (~10-20%)
Overfitting	Model memorizes training data, fails on new data
Underfitting	Model too simple, fails on both train and test data
Bias	Error from wrong assumptions in the learning algorithm
Variance	Error from sensitivity to fluctuations in training data

Bias-Variance Tradeoff:



 Code

```
python
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Load example data
from sklearn.datasets import load_iris
data = load_iris()
X, y = data.data, data.target

# Split data
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.5, random_state=42)

print(f"Train: {X_train.shape}, Val: {X_val.shape}, Test: {X_test.shape}")
# Train: (105, 4), Val: (22, 4), Test: (23, 4)
```

2. Data Preprocessing

Theory

Why Preprocessing? Raw data is messy — missing values, different scales, categorical variables. Models require clean, numerical, and well-scaled data.

Key Steps:

Raw Data

- |
- | — Handle Missing Values
 - | — Drop rows/columns
 - | — Impute (mean/median/mode)
 - | — Predict missing values (KNN Imputer)
- |
- | — Encode Categorical Variables
 - | — Label Encoding (Ordinal: Small→1, Medium→2, Large→3)
 - | — One-Hot Encoding (Nominal: Red=[1,0,0], Blue=[0,1,0])
- |
- | — Scale/Normalize Features
 - | — StandardScaler: $z = (x - \mu) / \sigma \rightarrow \text{mean}=0, \text{std}=1$
 - | — MinMaxScaler: $z = (x - \min) / (\max - \min) \rightarrow [0, 1]$
 - | — RobustScaler: $z = (x - \text{median}) / \text{IQR} \rightarrow \text{outlier-resistant}$
- |
- | — Handle Outliers
 - | — Z-score method ($|z| > 3$)
 - | — IQR method (outside $Q1-1.5*IQR, Q3+1.5*IQR$)

When to use which scaler:

Scaler	Use When
StandardScaler	Algorithm assumes Gaussian distribution (SVM, LR, PCA)
MinMaxScaler	Need values in [0,1], no outliers (Neural Networks)
RobustScaler	Data has many outliers
None	Tree-based models (Random Forest, XGBoost)

Code

python

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder, OneHotEncoder
from sklearn.impute import SimpleImputer

# — Create sample messy dataframe —————
df = pd.DataFrame({
    'age': [25, None, 35, 40, 22, None],
    'salary': [50000, 60000, None, 80000, 45000, 70000],
    'dept': ['IT', 'HR', 'IT', 'Finance', 'HR', 'Finance'],
    'target': [1, 0, 1, 0, 1, 0]
})

# — Handle Missing Values —————
# Strategy: mean for numerical
imputer = SimpleImputer(strategy='mean')
df[['age', 'salary']] = imputer.fit_transform(df[['age', 'salary']])

# — Encode Categorical —————
# One-Hot Encoding
df = pd.get_dummies(df, columns=['dept'], drop_first=True)
# Creates: dept_HR, dept_IT (Finance is reference)

# — Scale Features —————
scaler = StandardScaler()
feature_cols = ['age', 'salary']
df[feature_cols] = scaler.fit_transform(df[feature_cols])

print(df)

# — Handle Outliers with IQR —————
def remove_outliers_iqr(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return df[(df[col] >= lower) & (df[col] <= upper)]

```

3. Supervised Learning

3.1 Linear Regression

Theory

Predicts a **continuous output** by fitting a line (or hyperplane) through data.

Hypothesis: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$

Cost Function (MSE): $J(\theta) = (1/2m) \sum (\hat{y}_i - y_i)^2$

Gradient Descent: $\theta_j := \theta_j - \alpha \cdot \partial J / \partial \theta_j$

(Update weights in opposite direction of gradient)

Normal Equation: $\theta = (X^T X)^{-1} X^T y$ (closed-form, no iteration needed)

Code

```
python

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Generate data
np.random.seed(42)
X = np.random.randn(100, 1) * 10
y = 3 * X.squeeze() + 7 + np.random.randn(100) * 5

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Train
model = LinearRegression()
model.fit(X_train, y_train)

# Predict & Evaluate
y_pred = model.predict(X_test)
print(f'Intercept: {model.intercept_:.2f}')    # ≈ 7
print(f'Coefficient: {model.coef_[0]:.2f}')    # ≈ 3
print(f'MSE: {mean_squared_error(y_test, y_pred):.2f}')
print(f'R²: {r2_score(y_test, y_pred):.4f}')    # Closer to 1 = better
```

3.2 Logistic Regression

Theory

Despite the name, it's a **classification** algorithm. Applies sigmoid to linear output.

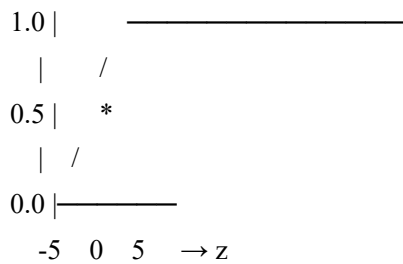
Sigmoid Function: $\sigma(z) = 1 / (1 + e^{-z}) \rightarrow \text{Output in } (0, 1)$

Hypothesis: $P(y=1|x) = \sigma(\theta^T x)$

Decision Boundary: Predict 1 if $P > 0.5$, else 0

Cost Function (Log Loss): $J(\theta) = -(1/m) \sum [y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$

Sigmoid Shape:



Code

```
python

from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_breast_cancer

# Load data
data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale (important for LR)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test) # Note: only transform, never fit on test!

# Train
clf = LogisticRegression(C=1.0, max_iter=1000)
clf.fit(X_train, y_train)

# Predict probabilities
probs = clf.predict_proba(X_test)[:, 1] # P(y=1)
preds = clf.predict(X_test)
print(f'Accuracy: {clf.score(X_test, y_test):.4f}')
```

3.3 Decision Trees

Theory

Splits data recursively based on feature thresholds to maximize **information gain** or minimize **Gini impurity**.

```
[Age <= 30?]  
 /      \  
[Yes]    [No]  
[Salary > 50K?] [Dept = IT?]  
 /  \    /  \  
High Risk Low Risk High Risk Low Risk
```

Gini Impurity: $G = 1 - \sum p_i^2$

Information Gain: $IG = H(\text{parent}) - \sum (|\text{child}|/|\text{parent}|) \cdot H(\text{child})$

$H(\text{entropy}) = -\sum p_i \cdot \log_2(p_i)$

Stopping Criteria: max_depth, min_samples_split, min_samples_leaf

Code

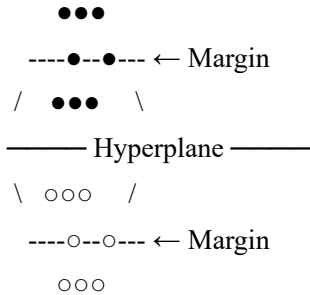
```
python  
  
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree  
  
clf = DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_split=10,  
    min_samples_leaf=5,  
    criterion='gini'  
)  
clf.fit(X_train, y_train)  
  
# Visualize rules  
print(export_text(clf, feature_names=data.feature_names.tolist()))  
  
# Feature importance  
importances = pd.Series(clf.feature_importances_, index=data.feature_names)  
print(importances.sort_values(ascending=False).head(5))  
  
# Plot tree  
plt.figure(figsize=(20, 8))  
plot_tree(clf, feature_names=data.feature_names, class_names=data.target_names,  
          filled=True, rounded=True, max_depth=2)  
plt.show()
```

3.4 Support Vector Machines (SVM)

Theory

Finds the **maximum margin hyperplane** that separates classes. Uses kernel trick for non-linear data.

Linear SVM:



Support Vectors: Points closest to the hyperplane

Objective: Maximize margin = $2/\|w\|$

Kernels:

- Linear: $K(x,z) = x^T z$
- Polynomial: $K(x,z) = (x^T z + c)^d$
- RBF/Gaussian: $K(x,z) = \exp(-\gamma \|x-z\|^2)$ ← most popular

C parameter: Small C = wider margin (more misclassification allowed). Large C = narrow margin (less misclassification).

Code

```
python

from sklearn.svm import SVC

svm = SVC(
    kernel='rbf',    # 'linear', 'poly', 'rbf', 'sigmoid'
    C=1.0,          # Regularization: smaller = more regularized
    gamma='scale',  # Kernel coefficient: 'scale', 'auto', or float
    probability=True # Enable predict_proba
)

svm.fit(X_train, y_train)
print(f'SVM Accuracy: {svm.score(X_test, y_test):.4f}')
print(f'Support Vectors: {svm.n_support_}') # Per class
```

3.5 K-Nearest Neighbors (KNN)

Theory

Classifies by majority vote of K nearest neighbors. No training phase — lazy learner.

Distance Metrics:

- Euclidean: $d = \sqrt{\sum (x_i - y_i)^2}$ ← most common
- Manhattan: $d = \sum |x_i - y_i|$
- Minkowski: $d = (\sum |x_i - y_i|^p)^{1/p}$

K=1 → very flexible, high variance

K=N → predict majority class, high bias

Optimal K usually $\sqrt{n_samples}$

Code

```
python

from sklearn.neighbors import KNeighborsClassifier

# Find optimal K
from sklearn.model_selection import cross_val_score

k_scores = []
K_range = range(1, 31)
for k in K_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='accuracy')
    k_scores.append(scores.mean())

optimal_k = K_range[np.argmax(k_scores)]
print(f"Optimal K: {optimal_k}")

knn = KNeighborsClassifier(n_neighbors=optimal_k)
knn.fit(X_train, y_train)
print(f"KNN Accuracy: {knn.score(X_test, y_test):.4f}")
```

4. Unsupervised Learning

4.1 K-Means Clustering

Theory

Partitions data into K clusters by minimizing **within-cluster sum of squares (WCSS)**.

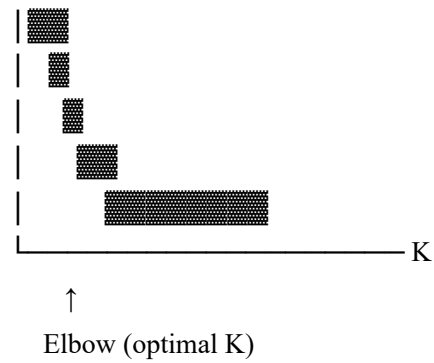
Algorithm:

1. Randomly initialize K centroids
2. Assign each point to nearest centroid
3. Recompute centroids as cluster means
4. Repeat steps 2-3 until convergence

Objective: $J = \sum_k \sum_{i \in C_k} \|x_i - \mu_k\|^2$

Elbow Method to find K:

WCSS



Code

python

```

from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate clustered data
X, true_labels = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

# Elbow Method
wcss = []
K_range = range(1, 11)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    wcss.append(km.inertia_)

# Plot elbow
plt.plot(K_range, wcss, 'bo-')
plt.xlabel('K'); plt.ylabel('WCSS'); plt.title('Elbow Method')
plt.show()

# Fit with optimal K=4
km = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = km.fit_predict(X)

# Visualize
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1],
            c='red', marker='*', s=300, label='Centroids')
plt.legend(); plt.show()

```

4.2 Principal Component Analysis (PCA)

Theory

Reduces dimensionality by projecting data onto directions of **maximum variance** (principal components).

Steps:

1. Standardize data (mean=0, std=1)
2. Compute Covariance Matrix: $C = X^T X / (n-1)$
3. Compute Eigenvectors and Eigenvalues of C
4. Sort by eigenvalue (descending)
5. Select top k eigenvectors → Principal Components
6. Project data: $Z = X \cdot W$ (W = matrix of top-k eigenvectors)

Explained Variance: $EV_i = \lambda_i / \sum \lambda_j$

Cumulative Variance: Choose k where cumulative EV > 95%

2D PCA Visualization:

Original Space

PCA Space

●●●●●●●●●● → ●●●●●●●●●● (1st PC, max variance)
● features ● ●●●●● (2nd PC, remaining variance)



Code

python

```

from sklearn.decomposition import PCA

# Fit PCA
pca = PCA()
pca.fit(X_train)

# Explained variance ratio
cumvar = np.cumsum(pca.explained_variance_ratio_)
n_components = np.argmax(cumvar >= 0.95) + 1
print(f'Components for 95% variance: {n_components}')

# Plot scree plot
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.bar(range(1, len(pca.explained_variance_ratio_)+1), pca.explained_variance_ratio_)
plt.xlabel('Component'); plt.ylabel('Explained Variance'); plt.title('Scree Plot')

plt.subplot(1, 2, 2)
plt.plot(cumvar, 'b-o')
plt.axhline(y=0.95, color='r', linestyle='--', label='95% threshold')
plt.xlabel('Components'); plt.ylabel('Cumulative Variance'); plt.legend()
plt.tight_layout(); plt.show()

# Apply PCA
pca = PCA(n_components=2) # For visualization
X_2d = pca.fit_transform(X_train)
plt.scatter(X_2d[:, 0], X_2d[:, 1], c=y_train, cmap='Set1')
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.title('PCA 2D Projection')
plt.show()

```

4.3 DBSCAN

Theory

Density-Based Spatial Clustering — finds clusters of arbitrary shape, handles noise/outliers.

Parameters:

- eps (ϵ): Neighborhood radius
- min_samples: Min points to form a dense region

Point Types:

- Core point: Has $\geq$ min_samples within ϵ
- Border point: Within ϵ of a core point, but not core itself
- Noise point: Neither core nor border $\rightarrow$ labeled -1

Advantage over K-Means:

- No need to specify K
- Finds non-spherical clusters
- Identifies outliers automatically

Code

```
python

from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, _ = make_moons(n_samples=200, noise=0.1, random_state=42)

db = DBSCAN(eps=0.3, min_samples=5)
labels = db.fit_predict(X)

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
n_noise = list(labels).count(-1)
print(f'Clusters: {n_clusters}, Noise points: {n_noise}')

# Visualize
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='Paired')
plt.title(f'DBSCAN: {n_clusters} clusters, {n_noise} noise points')
plt.show()
```

5. Model Evaluation & Metrics

Theory

Classification Metrics:

Confusion Matrix:

Predicted

Positive Negative

Actual Positive | TP | FN | ← False Negative (missed)

Negative | FP | TN | ← False Positive (false alarm)

Accuracy = $(TP + TN) / \text{Total}$ ← misleading with imbalanced data

Precision = $TP / (TP + FP)$ ← "Of predicted positives, how many are right?"

Recall = $TP / (TP + FN)$ ← "Of actual positives, how many did we catch?"

F1 Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

ROC-AUC = Area under Receiver Operating Characteristic curve

When to use:

- Precision: Spam detection (FP = annoying but OK)
- Recall: Cancer detection (FN = dangerous!)
- F1: Balanced, imbalanced datasets
- Accuracy: Only when classes are balanced

Regression Metrics:

MAE = $(1/n) \sum |y_i - \hat{y}_i|$ ← Average absolute error, robust to outliers

MSE = $(1/n) \sum (y_i - \hat{y}_i)^2$ ← Penalizes large errors more

RMSE = $\sqrt{\text{MSE}}$ ← Same units as target

$R^2 = 1 - (\text{SS}_{\text{res}} / \text{SS}_{\text{tot}})$ ← 1 = perfect, 0 = baseline mean model, can be negative

Code

python

```

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, roc_auc_score, roc_curve
)
import seaborn as sns

# Assume clf is a trained classifier
y_pred = clf.predict(X_test)
y_prob = clf.predict_proba(X_test)[:, 1]

# Classification Report
print(classification_report(y_test, y_pred, target_names=['Benign', 'Malignant']))

# Confusion Matrix Heatmap
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Benign', 'Malignant'],
            yticklabels=['Benign', 'Malignant'])
plt.ylabel('Actual'); plt.xlabel('Predicted'); plt.title('Confusion Matrix')
plt.show()

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)
plt.plot(fpr, tpr, label=f'AUC = {auc:.3f}')
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate'); plt.ylabel('True Positive Rate')
plt.title('ROC Curve'); plt.legend(); plt.show()

# Cross-Validation
from sklearn.model_selection import cross_val_score
cv_scores = cross_val_score(clf, X, y, cv=5, scoring='f1')
print(f'CV F1: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}')

```

6. Feature Engineering

Theory

Feature engineering is often more impactful than model choice.

Techniques:

Polynomial Features	$x_1, x_2 \rightarrow x_1^2, x_2^2, x_1x_2$
Log Transform	Skewed $\rightarrow \log(x+1)$
Binning	Continuous $\rightarrow$ Categories
Interaction Features	$x_1 \times x_2$
Aggregations	mean, std, min, max
Date Features	hour, day, month, weekday
Target Encoding	Category $\rightarrow$ mean(target)

Feature Selection Methods:

- Filter: Correlation, Chi², ANOVA F-test (fast, model-agnostic)
- Wrapper: RFE, Forward/Backward selection (slow, model-specific)
- Embedded: L1 Regularization, Tree feature importance (built into model)



Code

python

```

from sklearn.preprocessing import PolynomialFeatures
from sklearn.feature_selection import SelectKBest, f_classif, RFE
import pandas as pd

# — Polynomial Features —————
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_train)
print(f'Original features: {X_train.shape[1]}, Poly features: {X_poly.shape[1]}')

# — Feature Selection: Statistical (Filter) —————
selector = SelectKBest(score_func=f_classif, k=10)
X_selected = selector.fit_transform(X_train, y_train)
selected_mask = selector.get_support()
selected_features = np.array(data.feature_names)[selected_mask]
print("Top 10 features:", selected_features)

# — Recursive Feature Elimination (Wrapper) —————
from sklearn.linear_model import LogisticRegression
rfe = RFE(estimator=LogisticRegression(max_iter=1000), n_features_to_select=5)
rfe.fit(X_train, y_train)
print("RFE selected features:", np.array(data.feature_names)[rfe.support_])

# — Feature Importance from Tree (Embedded) —————
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
importance_df = pd.DataFrame({
    'feature': data.feature_names,
    'importance': rf.feature_importances_
}).sort_values('importance', ascending=False)
print(importance_df.head(10))

# — Log Transform for skewed features —————
# Use when data is right-skewed
X_log = np.log1p(X) # log(x+1) to handle zeros

```

7. Regularization

Theory

Regularization adds a **penalty term** to the cost function to prevent overfitting by discouraging large weights.

Original Cost: $J(\theta) = \text{MSE or Log-Loss}$

L1 (Lasso): $J(\theta) = \text{original} + \lambda \sum |\theta_j|$

→ Sparse solution (many weights → exactly 0)

→ Good for feature selection

L2 (Ridge): $J(\theta) = \text{original} + \lambda \sum \theta_j^2$

→ Small weights, but none exactly 0

→ Good when all features somewhat relevant

Elastic Net: $J(\theta) = \text{original} + \lambda_1 \sum |\theta_j| + \lambda_2 \sum \theta_j^2$

→ Combination of L1 and L2

Effect of λ (regularization strength):

Small λ → Less regularization → Risk overfitting

Large λ → More regularization → Risk underfitting

Weight Visualization:

L1 (Lasso)



L2 (Ridge)



Diamond constraint =

corners → sparse solution

Circle constraint =

smooth → small weights

Code

python

```

from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=200, n_features=30, noise=15, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Ridge (L2)
ridge = Ridge(alpha=1.0) # alpha =  $\lambda$ 
ridge.fit(X_train, y_train)
print(f'Ridge R²: {ridge.score(X_test, y_test):.4f}')
print(f'Non-zero weights: {np.sum(ridge.coef_ != 0)}') # All weights

# Lasso (L1)
lasso = Lasso(alpha=1.0)
lasso.fit(X_train, y_train)
print(f'Lasso R²: {lasso.score(X_test, y_test):.4f}')
print(f'Non-zero weights: {np.sum(lasso.coef_ != 0)}') # Sparse!

# Elastic Net
en = ElasticNet(alpha=1.0, l1_ratio=0.5) # l1_ratio: 0=Ridge, 1=Lasso
en.fit(X_train, y_train)
print(f'ElasticNet R²: {en.score(X_test, y_test):.4f}')

# Alpha search
alphas = np.logspace(-3, 3, 100)
ridge_scores = [Ridge(alpha=a).fit(X_train, y_train).score(X_test, y_test) for a in alphas]
best_alpha = alphas[np.argmax(ridge_scores)]
print(f'Best Ridge alpha: {best_alpha:.4f}')

```

8. Ensemble Methods

Theory

Ensemble = Combine multiple weak models → stronger model

Three Ensemble Strategies:

BAGGING	BOOSTING	STACKING
(Parallel)	(Sequential)	(Meta-learning)
Bootstrap samples	Each model fixes	Level-1 models
→ Train models	previous model's	predict → Level-2
→ Average/Vote	errors	model combines
Example:	Example:	Example:
Random Forest	AdaBoost	Blend LR+RF+SVM
	Gradient Boosting	
	XGBoost, LightGBM	
	CatBoost	

Why Random Forest works:

- Bootstrap sampling introduces diversity (different training sets)
- Random feature selection at each split (prevents correlation)
- Averaging reduces variance without increasing bias

Code

```
python
```

```
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier,
    AdaBoostClassifier, VotingClassifier, BaggingClassifier
)
import xgboost as xgb
```

```
# — Random Forest —————
```

```
rf = RandomForestClassifier(
    n_estimators=200,    # Number of trees
    max_depth=None,     # Grow full trees
    max_features='sqrt', # Features per split (classification default)
    min_samples_leaf=1,
    bootstrap=True,
    random_state=42
)
rf.fit(X_train, y_train)
print(f'Random Forest: {rf.score(X_test, y_test):.4f}')
```

```
# — Gradient Boosting —————
```

```
gb = GradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.1,    # Shrinkage factor (smaller = more robust)
    max_depth=3,
    subsample=0.8,        # Fraction of samples per tree
    random_state=42
)
gb.fit(X_train, y_train)
print(f'Gradient Boosting: {gb.score(X_test, y_test):.4f}')
```

```
# — XGBoost —————
```

```
xgb_clf = xgb.XGBClassifier(
    n_estimators=200,
    learning_rate=0.1,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8, # Feature fraction per tree
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)
xgb_clf.fit(X_train, y_train,
            eval_set=[(X_test, y_test)],
            early_stopping_rounds=10, # Stop if no improvement
            verbose=False)
print(f'XGBoost: {xgb_clf.score(X_test, y_test):.4f}')
```

```
# — Voting Ensemble —
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

voting = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(max_iter=1000)),
        ('rf', RandomForestClassifier(n_estimators=100)),
        ('svm', SVC(probability=True))
    ],
    voting='soft' # 'hard' = majority vote, 'soft' = weighted probabilities
)

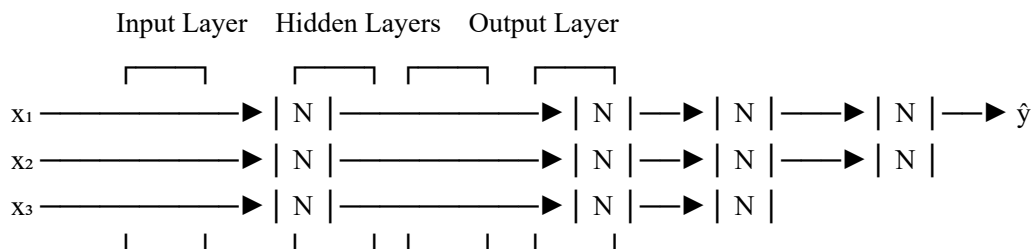
voting.fit(X_train, y_train)
print(f'Voting Ensemble: {voting.score(X_test, y_test):.4f}')
```

9. Neural Networks & Deep Learning

Theory

Neural networks are composed of **layers of neurons** that learn hierarchical representations.

Architecture:



Each neuron: $z = \sum(w_i x_i) + b$, $a = \text{activation}(z)$

Activation Functions:

- Sigmoid: $\sigma(z) = 1/(1+e^{-z}) \rightarrow \text{output } [0,1]$, vanishing gradient
- Tanh: $\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}) \rightarrow \text{output } [-1,1]$
- ReLU: $f(z) = \max(0, z) \rightarrow \text{most popular, fast}$
- Leaky ReLU: $f(z) = \max(0.01z, z) \rightarrow \text{fixes dying ReLU}$
- Softmax: $e^{z_i} / \sum e^{z_j} \rightarrow \text{output layer (multi-class)}$

Backpropagation: Chain rule to compute gradients layer by layer

Optimizer: Adam, SGD, RMSProp — update weights using gradients

When to use Deep Learning:

Use DL when:

- Large amounts of data (>100K samples)
- Unstructured data (images, text, audio)
- Complex patterns with many features

Use traditional ML when:

- Small datasets (<10K samples)
- Structured/tabular data
- Interpretability required
- Limited compute



Code

```
python
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
```

```
# — Define Neural Network —————
```

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dims, output_dim, dropout=0.3):
        super(MLP, self).__init__()
        layers = []
        dims = [input_dim] + hidden_dims
        for i in range(len(dims) - 1):
            layers.extend([
                nn.Linear(dims[i], dims[i+1]),
                nn.BatchNorm1d(dims[i+1]),    # Normalize activations
                nn.ReLU(),
                nn.Dropout(dropout)          # Regularization
            ])
        layers.append(nn.Linear(dims[-1], output_dim))
        self.network = nn.Sequential(*layers)

    def forward(self, x):
        return self.network(x)
```

```
# — Prepare Data —————
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

data = load_breast_cancer()
X, y = data.data.astype(np.float32), data.target.astype(np.float32)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Convert to tensors
```

```
X_train_t = torch.tensor(X_train, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_test_t = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
```

```
train_loader = DataLoader(TensorDataset(X_train_t, y_train_t), batch_size=32, shuffle=True)
```

```
# — Initialize Model —————
```

```
model = MLP(
```

```

input_dim=X_train.shape[1],
hidden_dims=[64, 32, 16],
output_dim=1,
dropout=0.3
)
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)
criterion = nn.BCEWithLogitsLoss()

# — Training Loop —————
def train_model(model, loader, optimizer, criterion, n_epochs=50):
    train_losses = []
    for epoch in range(n_epochs):
        model.train()
        epoch_loss = 0
        for X_batch, y_batch in loader:
            optimizer.zero_grad()
            output = model(X_batch)
            loss = criterion(output, y_batch)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item()

        if (epoch + 1) % 10 == 0:
            model.eval()
            with torch.no_grad():
                test_out = model(X_test_t)
                test_loss = criterion(test_out, y_test_t).item()
                test_acc = ((torch.sigmoid(test_out) > 0.5).float() == y_test_t.float()).mean()
            print(f'Epoch {epoch+1}/{n_epochs} | Train Loss: {epoch_loss/len(loader):.4f} '
                  f'| Test Loss: {test_loss:.4f} | Test Acc: {test_acc:.4f}')
            train_losses.append(epoch_loss)
    return train_losses

losses = train_model(model, train_loader, optimizer, criterion, n_epochs=50)

```

9.1 Convolutional Neural Networks (CNN) — Quick Reference

Input Image $\rightarrow$ [Conv $\rightarrow$ ReLU $\rightarrow$ Pool] $\times$ N $\rightarrow$ Flatten $\rightarrow$ FC Layers $\rightarrow$ Output

Conv Layer: Slides filters over image, extracts local patterns

MaxPool: Reduces spatial dimensions (downsampling)

Flatten: 2D feature maps $\rightarrow$ 1D vector for FC layers

```

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1), # (B, 1, 28, 28) → (B, 32, 28, 28)
            nn.ReLU(),
            nn.MaxPool2d(2), # → (B, 32, 14, 14)
            nn.Conv2d(32, 64, kernel_size=3, padding=1), # → (B, 64, 14, 14)
            nn.ReLU(),
            nn.MaxPool2d(2), # → (B, 64, 7, 7)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        return self.classifier(self.features(x))

```

10. Natural Language Processing (NLP)

Theory

NLP Pipeline:

Raw Text → Tokenize → Clean → Vectorize → Model → Output

Text Cleaning:

- Lowercasing, punctuation removal
- Stop words removal ("the", "is", "at")
- Stemming: "running" → "run" (crude)
- Lemmatization: "running" → "run" (proper)

Vectorization:

Bag of Words	Word counts, ignores order
TF-IDF	Word importance = freq × rarity
Word2Vec	Dense 300-dim embeddings
GloVe	Global statistics + local context
BERT	Contextual embeddings (transformer)

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

$$\text{TF} = (\text{term freq in doc}) / (\text{total terms in doc})$$

$$\text{IDF} = \log(\text{total docs} / \text{docs with term})$$



Code

```
python
```

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import MultinomialNB
import re
```

```
# Sample corpus
```

```
texts = [
    "Machine learning is amazing for data science",
    "Deep learning neural networks are powerful",
    "Python is great for machine learning",
    "Data science requires statistics and coding",
    "Neural networks can learn complex patterns"
]
```

```
labels = [1, 1, 1, 0, 1]
```

```
# — Text Cleaning Function —————
```

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

```
cleaned = [clean_text(t) for t in texts]
```

```
# — TF-IDF Vectorization —————
```

```
tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),    # Unigrams and bigrams
    stop_words='english',
    min_df=1,
    max_df=0.95
)
```

```
X_tfidf = tfidf.fit_transform(cleaned)
```

```
print(f"Vocab size: {len(tfidf.vocabulary_)}")
```

```
# — Classification Pipeline —————
```

```
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(ngram_range=(1, 2), stop_words='english')),
    ('clf', MultinomialNB(alpha=0.1))
])
```

```
pipeline.fit(cleaned, labels)
```

```
new_text = ["deep learning with python is incredible"]
```

```
print(f"Prediction: {pipeline.predict(new_text)}")
```

```
# — Word Embeddings with transformers (Hugging Face) —————
```

```
# pip install transformers
```

```

from transformers import AutoTokenizer, AutoModel
import torch

tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModel.from_pretrained('bert-base-uncased')

inputs = tokenizer("Machine learning is great!", return_tensors='pt')
with torch.no_grad():
    outputs = model(**inputs)
# CLS token embedding (sentence representation)
embedding = outputs.last_hidden_state[:, 0, :] # Shape: (1, 768)
print(f"BERT embedding shape: {embedding.shape}")

```

11. Hyperparameter Tuning

Theory

Types of Parameters:

- Parameters: Learned from data (weights, biases)
- Hyperparameters: Set before training (learning rate, n_estimators, depth)

Search Strategies:

Grid Search	Exhaustive search over specified values
	Pros: Finds exact best combo
	Cons: Exponential time complexity
Random Search	Sample random combinations
	Pros: More efficient, finds good configs
	Cons: May miss optimal
Bayesian Optim. (Optuna/Hyperopt)	Use past results to pick next params
	Pros: Most efficient, intelligent search
	Cons: More complex setup

Code

```
python
```

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from scipy.stats import randint, uniform
```

```
# — Grid Search CV —————
```

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [3, 5, 7, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
grid_cv = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring='f1',
    n_jobs=-1,    # Use all CPU cores
    verbose=1
)
grid_cv.fit(X_train, y_train)
print(f'Best params: {grid_cv.best_params_}')
print(f'Best F1: {grid_cv.best_score_:.4f}')
```

```
# — Random Search CV —————
```

```
param_dist = {
    'n_estimators': randint(100, 500),
    'max_depth': [3, 5, 7, None],
    'min_samples_split': randint(2, 20),
    'min_samples_leaf': randint(1, 10),
    'max_features': uniform(0.1, 0.9)
}
random_cv = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_dist,
    n_iter=50,    # Number of random combinations to try
    cv=5,
    scoring='f1',
    n_jobs=-1,
    random_state=42
)
random_cv.fit(X_train, y_train)
print(f'Best params: {random_cv.best_params_}')
```

```
# — Optuna (Bayesian Optimization) —————
```

```
# pip install optuna
import optuna
optuna.logging.set_verbosity(optuna.logging.WARNING)
```

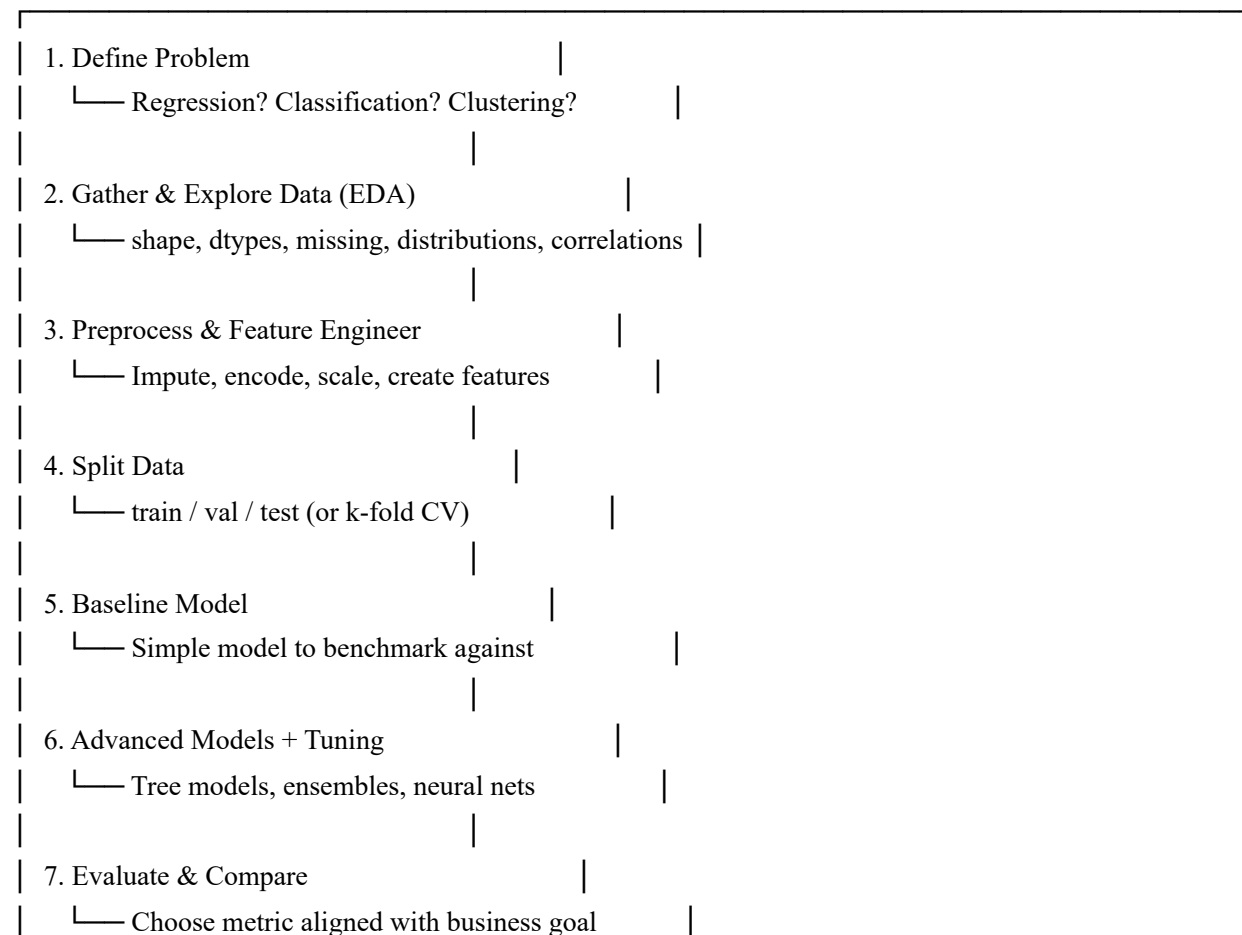
```
def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 100, 500),
        'max_depth': trial.suggest_int('max_depth', 2, 15),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3, log=True),
        'subsample': trial.suggest_float('subsample', 0.5, 1.0),
    }
    import xgboost as xgb
    model = xgb.XGBClassifier(**params, random_state=42, eval_metric='logloss', use_label_encoder=False)
    score = cross_val_score(model, X_train, y_train, cv=3, scoring='f1').mean()
    return score

study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
print(f'Best value: {study.best_value:.4f}')
print(f'Best params: {study.best_params}')
```

12. End-to-End ML Pipeline

Theory

Complete ML Pipeline:



8. Interpret Model

- SHAP values, feature importance, coefficients

9. Deploy & Monitor

- REST API, batch prediction, drift monitoring



Code — Full Pipeline Example

```
python
```

```
import pandas as pd
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import classification_report
import warnings
warnings.filterwarnings('ignore')
```

— 1. Load Data —————

```
df = pd.read_csv('your_data.csv') # Replace with actual data
```

Quick EDA

```
print("Shape:", df.shape)
print("\nMissing values:\n", df.isnull().sum())
print("\nData types:\n", df.dtypes)
print("\nTarget distribution:\n", df['target'].value_counts(normalize=True))
print("\nStatistics:\n", df.describe())
```

— 2. Feature Engineering —————

```
# (Add your domain-specific features here)
# df['new_feature'] = df['col1'] * df['col2']
```

— 3. Define Feature Types —————

```
target = 'target'
X = df.drop(columns=[target])
y = df[target]

numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X.select_dtypes(include=['object', 'category']).columns.tolist()
```

— 4. Build Preprocessing Pipelines —————

```
numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
categorical_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])
preprocessor = ColumnTransformer([
    ('num', numeric_pipe, numeric_cols),
    ('cat', categorical_pipe, categorical_cols)
```

```
)
```

— 5. Define Models —————

```
models = {  
    'Random Forest': RandomForestClassifier(n_estimators=200, random_state=42),  
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=200, random_state=42),  
}
```

— 6. Train-Test Split —————

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
                                                    random_state=42, stratify=y)
```

— 7. Cross-Validate All Models —————

```
results = {}  
for name, clf in models.items():  
    full_pipe = Pipeline([('prep', preprocessor), ('clf', clf)])  
    cv_scores = cross_val_score(full_pipe, X_train, y_train, cv=5,  
                                scoring='f1_weighted', n_jobs=-1)  
    results[name] = cv_scores  
    print(f'{name}: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}')
```

— 8. Train Best Model on Full Train Set —————

```
best_model_name = max(results, key=lambda k: results[k].mean())  
best_clf = models[best_model_name]  
final_pipeline = Pipeline([('prep', preprocessor), ('clf', best_clf)])  
final_pipeline.fit(X_train, y_train)
```

— 9. Final Evaluation —————

```
y_pred = final_pipeline.predict(X_test)  
print(f'\nBest Model: {best_model_name}')
```

```
print(classification_report(y_test, y_pred))
```

— 10. SHAP Interpretability —————

```
# pip install shap  
import shap  
X_train_prep = final_pipeline.named_steps['prep'].transform(X_train)  
explainer = shap.TreeExplainer(final_pipeline.named_steps['clf'])  
shap_values = explainer.shap_values(X_train_prep)  
shap.summary_plot(shap_values, X_train_prep)
```

— 11. Save Model —————

```
import joblib  
joblib.dump(final_pipeline, 'model.pkl')
```

Load and predict

```
loaded_model = joblib.load('model.pkl')  
predictions = loaded_model.predict(X_test)
```

— 12. Serve as API (Flask example) —

```
"""
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load('model.pkl')

@app.route('/predict', methods=['POST'])
def predict():
    data = request.get_json()
    df = pd.DataFrame([data])
    pred = model.predict(df)[0]
    prob = model.predict_proba(df)[0].max()
    return jsonify({'prediction': int(pred), 'confidence': float(prob)})

if __name__ == '__main__':
    app.run(debug=True)
"""
```

Quick Reference Tables

Algorithm Selection Guide

TASK → ALGORITHM

Regression:

Linear relationship	→ Linear Regression
Non-linear, small data	→ Polynomial Regression / SVR
Many features, sparse	→ Lasso Regression
Robust to outliers	→ Huber / Ridge Regression
Best accuracy	→ XGBoost / Random Forest

Classification:

Baseline	→ Logistic Regression
High dimensional, sparse	→ SVM (Linear)
Complex boundaries	→ SVM (RBF) / Random Forest
Probabilistic output needed	→ Logistic Regression / Naive Bayes
Best accuracy	→ XGBoost / LightGBM
Text/NLP	→ BERT / Naive Bayes / Logistic Reg.
Images	→ CNN (ResNet, EfficientNet)

Clustering:

Know number of clusters → K-Means
Unknown K, arbitrary shapes → DBSCAN
Hierarchy needed → Agglomerative Clustering

Dimensionality Reduction:
Linear projection → PCA
Non-linear visualization → t-SNE / UMAP

Common Libraries

Library	Purpose
numpy	Array math, linear algebra
pandas	Data manipulation (DataFrames)
matplotlib / seaborn	Visualization
scikit-learn	Classical ML algorithms
xgboost / lightgbm	Gradient boosting
pytorch / tensorflow	Deep learning
transformers	Pre-trained language models
optuna	Hyperparameter optimization
shap	Model interpretability
mlflow	Experiment tracking

Key Formulas Summary

Linear Regression: $\hat{y} = X\theta, J = (1/2m)||X\theta - y||^2$
Logistic Regression: $\hat{y} = \sigma(X\theta), J = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$
Gradient Descent: $\theta \leftarrow \theta - \alpha \nabla J(\theta)$
Ridge (L2): $J + \lambda ||\theta||^2$
Lasso (L1): $J + \lambda ||\theta||_1$
SVM Margin: $2/||w||, \text{ maximize } \leftrightarrow \text{ minimize } ||w||^2/2$
Gini Impurity: $1 - \sum p_i^2$
Information Gain: $H(\text{parent}) - \sum |\text{child}|/|\text{parent}| \cdot H(\text{child})$
K-Means Objective: $\sum_k \sum_{i \in C_k} ||x_i - \mu_k||^2$
PCA: $Z = XW, W = \text{top-k eigenvectors of } X^T X$
Accuracy: $(TP+TN)/(TP+TN+FP+FN)$
Precision: $TP/(TP+FP)$

Recall:	$TP/(TP+FN)$
F1:	$2 \cdot P \cdot R / (P+R)$

Setup & Installation

```
bash

# Create virtual environment
python -m venv ml-env
source ml-env/bin/activate # Linux/Mac
ml-env\Scripts\activate   # Windows

# Install core packages
pip install numpy pandas matplotlib seaborn scikit-learn
pip install xgboost lightgbm catboost
pip install torch torchvision # PyTorch
pip install transformers datasets # Hugging Face
pip install optuna shap mlflow joblib
pip install jupyter notebook

# Start Jupyter
jupyter notebook
```

Built for ML practitioners — from fundamentals to production. Happy learning! 🚀